MYRTOS

KERNEL TESTING & VALIDATION REPORT

Engineering validation report | ARM Cortex-M3 / STM32F103 | Keil µVision evidence

Document scope

This report consolidates six scenario-based tests of a custom educational MYRTOS kernel. Evidence consists of application and kernel source snapshots, Keil debugger views, and Logic Analyzer traces. Observations are distinguished from unverified timing, safety, and production-readiness claims.

Document status	Working draft — expandable
Validation method	Source-level debugging, register/watch inspection, and Keil Logic Analyzer
Current coverage	TC-01: stack setup, activation, startup, context switching and round robin; TC-02: priority ordering, high-priority activation, execution and self-termination logic.
Not covered by current tests	Quantitative timing fairness, worst-case latency, exhaustive state transitions and formal stress testing.

Test Case 01

Equal-Priority Round Robin & Context-Switching Walkthrough

Objective. Demonstrate creation and initialization of task-specific stacks; task activation and scheduler-table update; OS startup and SysTick-driven task selection; PendSV context-save/restore flow; processor execution mode; and observable alternating execution of three continuously running tasks.

Test setup	**Three application tasks: Task1, Task2, Task3; equal configured priority; each executes continuously in a while(1) loop without task delay/sleep.**
System task	Idle task present as part of kernel startup; not a fourth equal-priority application task.
Platform	STM32F103 Cortex-M3, MYRTOS kernel, Keil µVision debugger
Tick configuration	SysTick_Config(8000); 1 ms tick under the stated 8 MHz core-clock assumption.
Evidence	Four user-provided annotated figures; comments are illustrative and should be checked against live debugger values.

1. Task creation & initial stack frame

The task-creation sequence allocates a dedicated PSP stack region for each task and prepares its initial exception-return context. The debugger view is used to inspect stack boundaries, the task control structure, and initialized stack memory. This phase establishes task readiness for subsequent activation; it does not by itself prove a completed context switch.

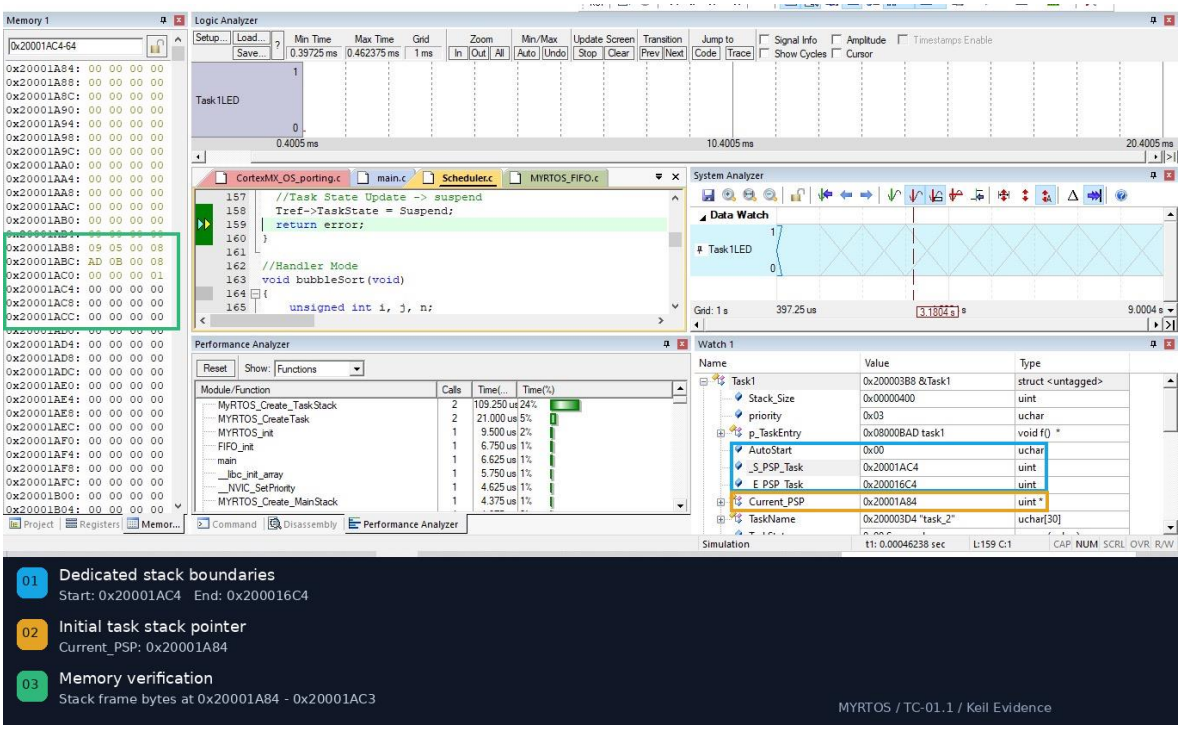


Figure 1. Task stack allocation and initial stack-frame inspection (annotated Keil evidence).

2. Activation & scheduler-table update

MYRTOS_ActivateTask() marks the requested task for activation and requests the corresponding kernel service. The scheduler update invokes priority sorting and updates the ready-task bookkeeping. The OS task array shows the current ordering of Task1, Task2, Task3, and the idle task. Equal-priority ordering in this test is consistent with the scenario; it is not a test of mixed-priority sorting.

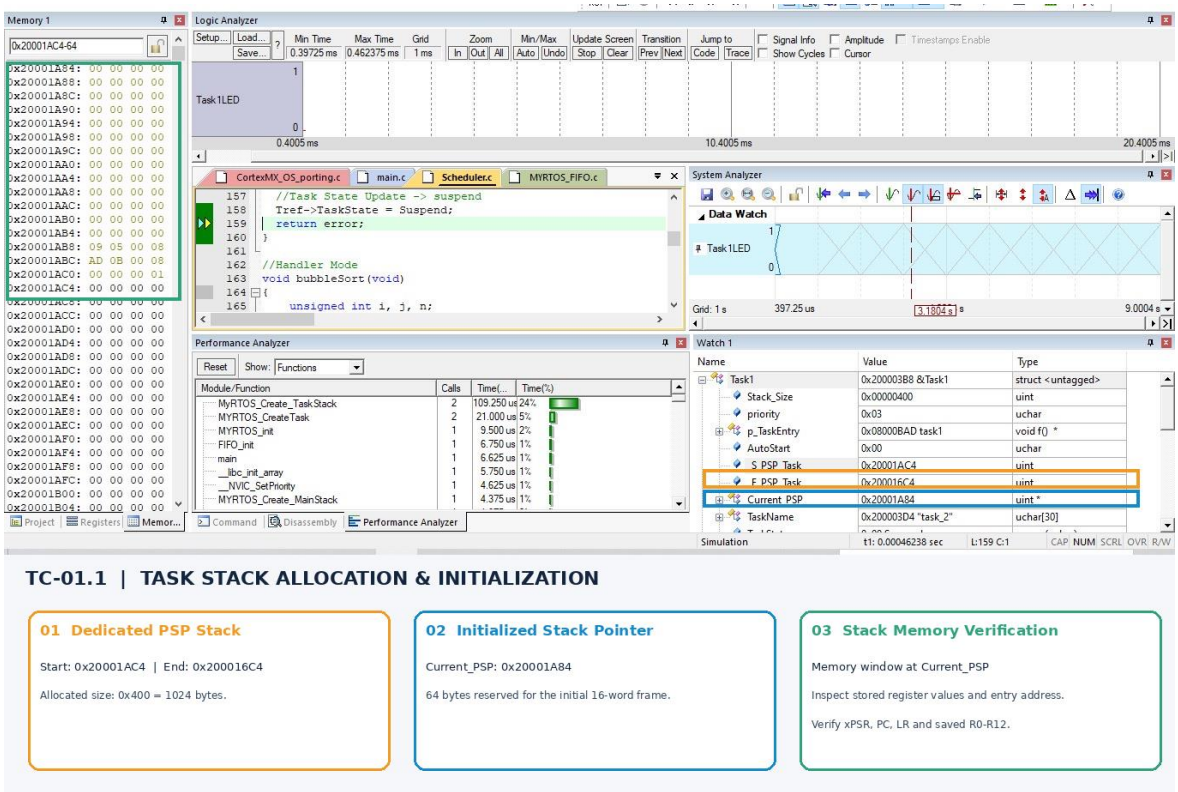


Figure 2. Activation request and scheduler-table update (annotated Keil evidence).

3. OS startup & scheduler decision

MYRTOS_STARTOS() starts the kernel, configures SysTick, sets up PSP-based Thread Mode execution, and invokes the idle-task entry path. On a SysTick interrupt, the kernel updates waiting-time bookkeeping, runs Decide_whatNext(), and requests PendSV. The debugger watch shows the selected NextTask, linking the scheduling decision to the subsequent exception.

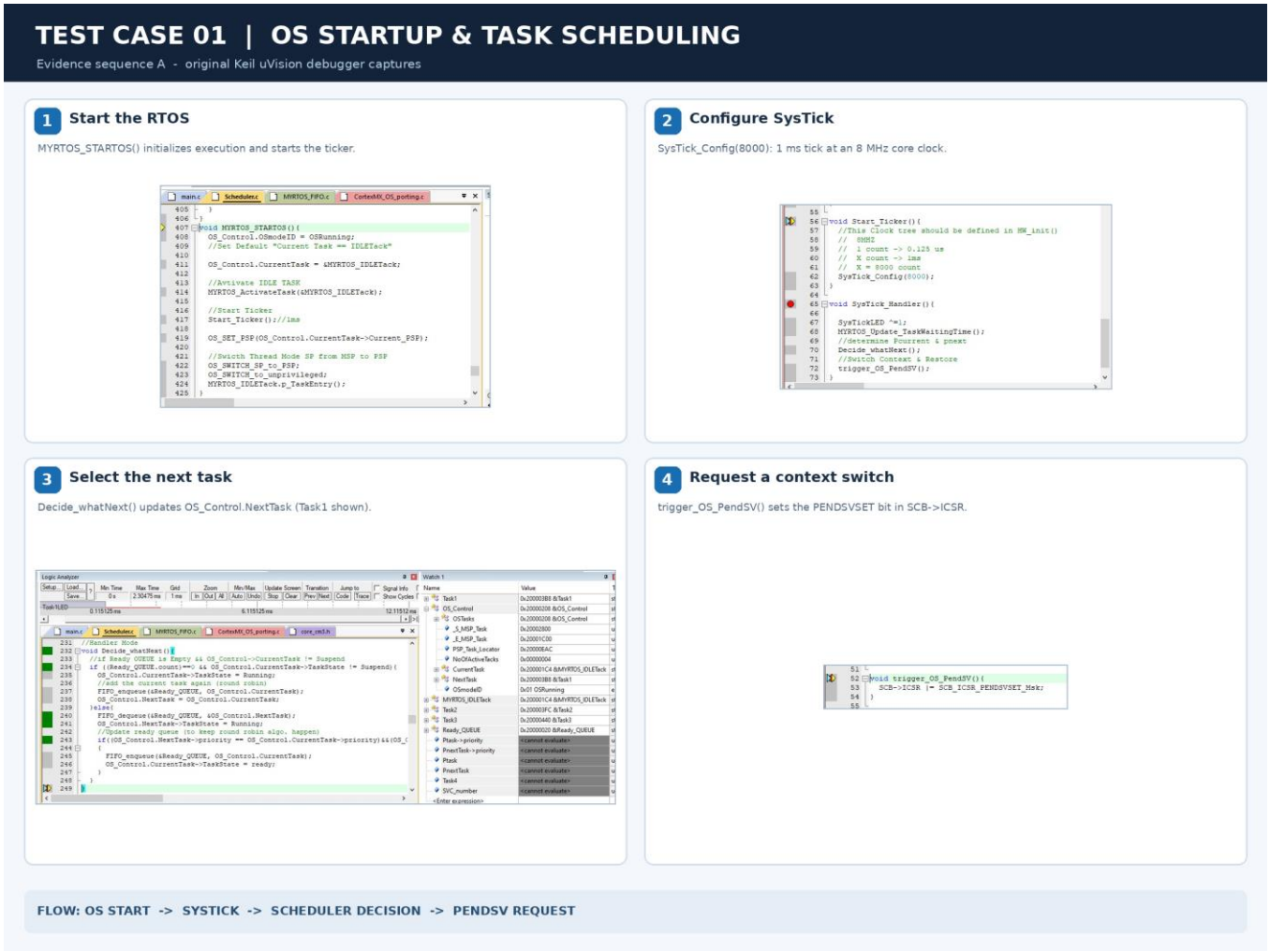


Figure 3. SysTick setup, scheduling decision, and PendSV request (annotated Keil evidence).

4. Context switch & execution observation

PendSV runs in privileged Handler Mode using MSP. The handler follows the context-save/context-restore path, including R4–R11 and task-specific PSP state. A separate debugger observation shows Thread Mode as unprivileged and using PSP. After exception return, execution reaches the selected task. Keil Logic Analyzer traces of Task1LED, Task2LED, and Task3LED show alternating task activity consistent with the intended equal-priority Round Robin scenario.

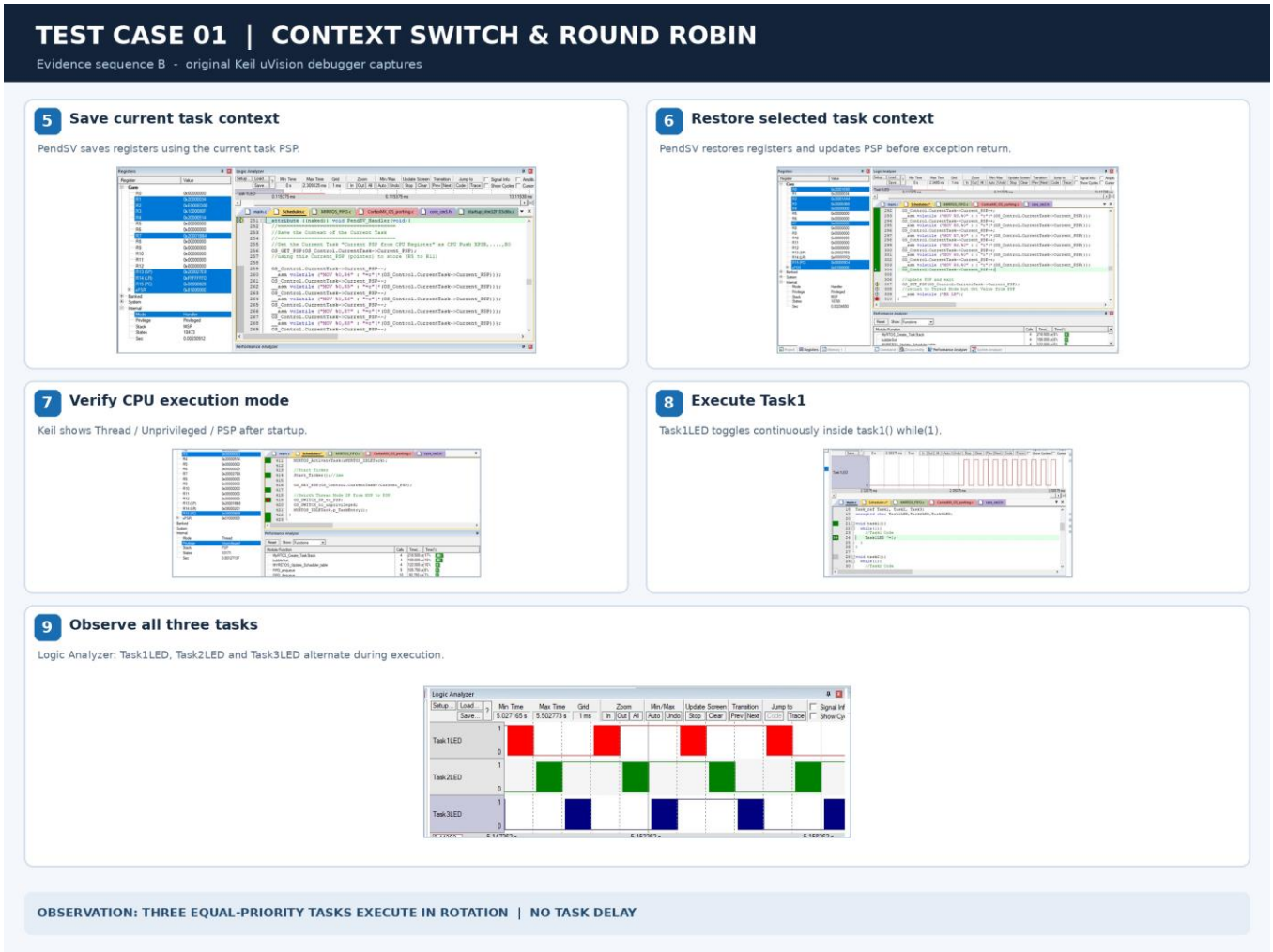


Figure 4. PendSV context handling, CPU mode observation, and alternating task activity.

Observed outcome & interpretation

Observed	**Three application-task indicators alternate in the captured Logic Analyzer view.**
Observed	Debugger captures show startup code, scheduler selection, PendSV handling, and processor-mode/stack observations.
Supported conclusion	The captures support a successful functional demonstration of task alternation and the intended context-switch path in this debug scenario.
Not yet established	Exact per-task CPU time, fairness bounds, robustness under all task states, and correctness across every interrupt/optimization configuration.

Engineering note. The annotated screenshots are report visuals based on the debugging session; where annotations contain inferred states or values, the original Keil watch/register readings remain authoritative. Full kernel correctness requires additional targeted tests.

Test Case 02

Priority-Based Preemption, Task Activation & Termination

Objective. Demonstrate that a task with a higher scheduling priority can be activated by Task1, selected for execution ahead of lower-priority ready tasks, and removed from execution after self-termination. This case focuses on scheduling behavior rather than repeating the low-level exception and register walkthrough from TC-01.

Test setup & trigger

Task1, Task2 and Task3 continue their equal-priority workload. Task4 has a higher scheduling priority (priority field 0x01 in the Keil Watch window) and is initially suspended. Task1 increments a local counter in its loop and calls MYRTOS_ActivateTask(&Task4) whenever Count reaches 30, then resets Count.

Evidence A — activation and priority ordering

The first Keil capture shows Task1 activation logic and Task4 in Suspend state. The second capture shows OS_Control.OSTasks ordered with Task4 at index [0], followed by Task1, Task2, Task3, and the idle task. Priority-table position alone does not make a suspended task runnable; Task4 must first be activated.

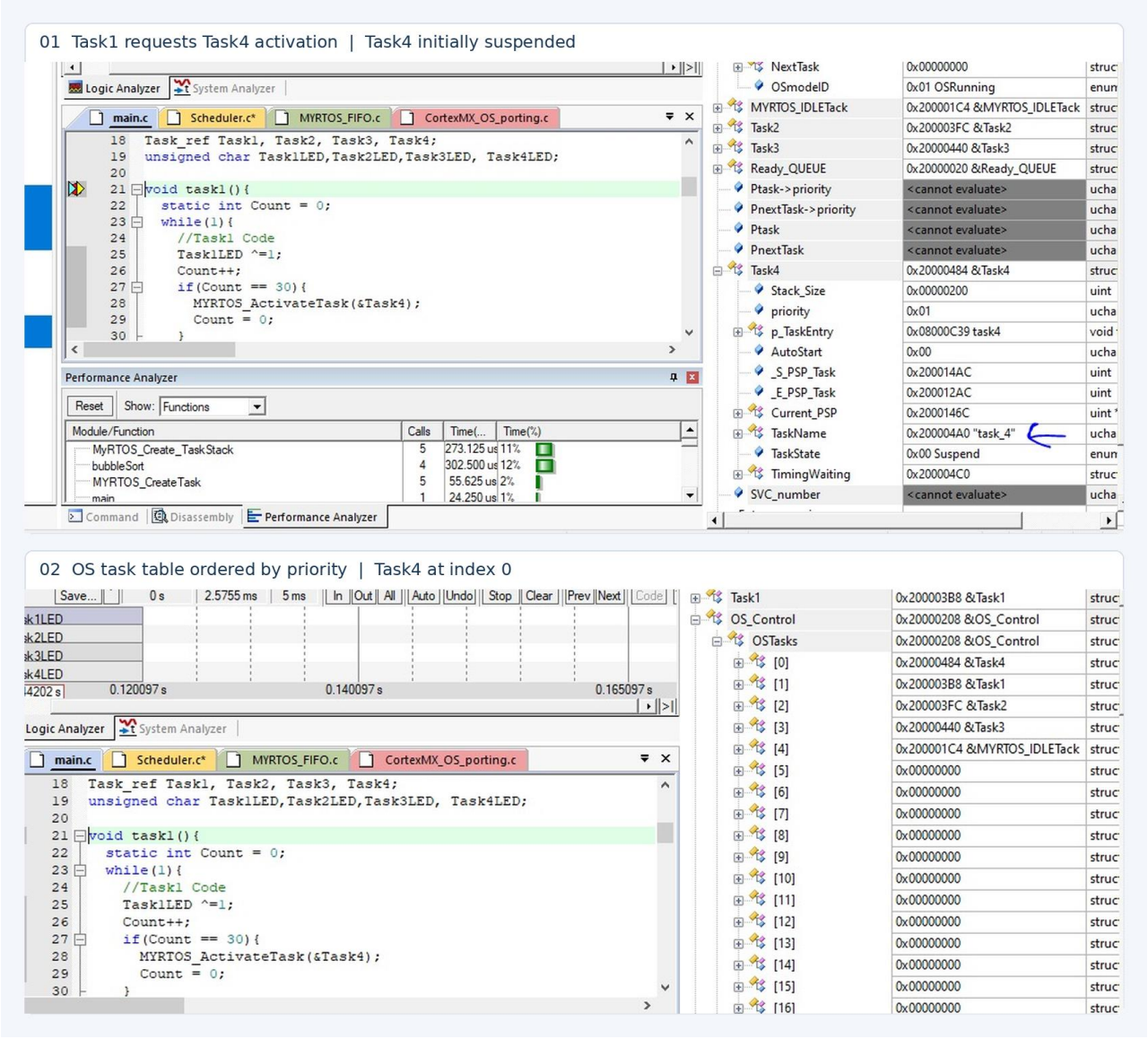


Figure 5. Original Keil evidence: Task4 activation request and OS task-table ordering.

Evidence B — Task4 execution and logic-analyzer observations

The third Keil capture shows Task4 with TaskState = Running and a loop that calls MYRTOS_TerminateTask(&Task4) when its counter reaches 0xFF. The fourth capture displays the activity traces of Task1LED, Task2LED, Task3LED, and Task4LED. Together, these observations support the higher-priority task activation and subsequent scheduling scenario.

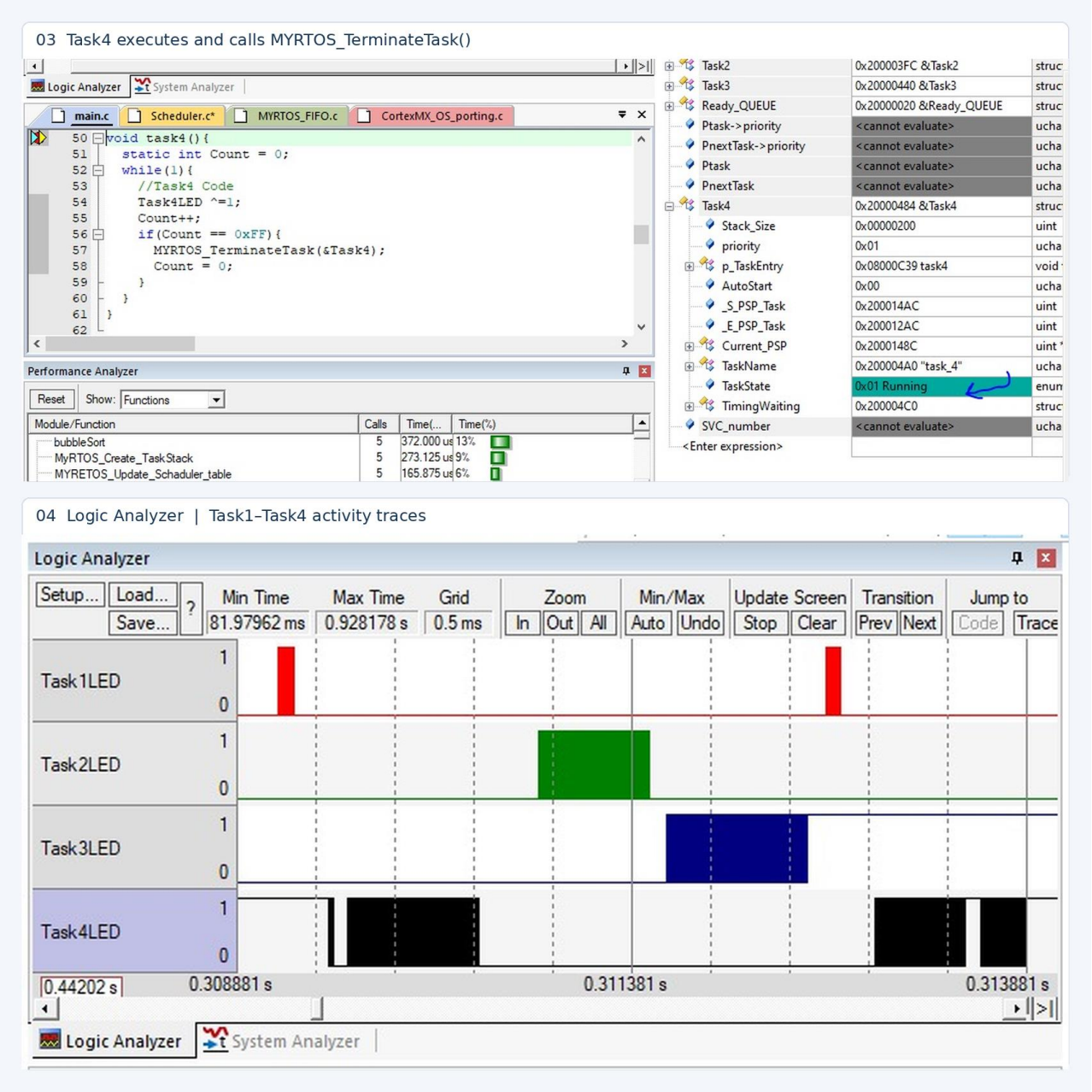


Figure 6. Original Keil evidence: Task4 execution, self-termination logic, and task activity traces.

Observed result & limits

Observed: Task4 appears in the priority-sorted OS task table while initially suspended; its code and Watch state later show Running; task-activity traces include Task4 alongside the other tasks. The code includes a self-termination request. Interpretation: the screenshots are consistent with priority-based activation, execution, and scheduling after termination. Exact preemption latency, per-task CPU time, and the post-termination state transition are not independently measured or directly shown by these captures.

Test Case 03

Tick-Based Task Delay, Suspension & Idle Execution

Objective. Validate the MYRTOS task-delay mechanism using a tick countdown, suspension of a delayed task, its transition back to the schedulable state after the requested interval, and execution of the idle task when no application task is runnable.

Test setup & sequence

Task1 toggles Task1LED and invokes MYRTOS_TaskWait(100, &Task1) within its loop. The 100-tick request corresponds nominally to 100 ms at the configured 1 ms SysTick rate; this test does not establish precise execution timing. Other task LED traces are observed in Keil.

1 Delay request: MYRTOS_TaskWait() enables TimingWaiting.Blocking, initializes TimingWaiting.Ticks_Count with NoTICKS, sets TaskState to Suspend, and requests scheduler processing through the existing termination-service path.

2 Tick maintenance: MYRTOS_Update_TaskWaitingTime() checks suspended tasks with blocking enabled, decrements their tick counters, and when a counter reaches zero disables blocking, changes the task state to Waiting, and requests a scheduler-table update.

3 Idle path: when the scheduler has no runnable application task, MYRTOS_idleTack() can run; its loop toggles idleTackLED and executes WFE (Wait For Event). WFE is an event-wait instruction, not by itself proof of a measured low-power saving.

Evidence A — delay request and countdown

1 Task1 requests a 100-tick delay

```
20
21 void task1(){
22   static int Count = 0;
23   while(1){
24     //Task1 Code
25     Task1LED ^=1;
26     Count++;
27     if(Count == 30){
28       MYRTOS_ActivateTask(&Task4);
29       Count = 0;
30     }//100ms
31     MYRTOS_TaskWait(100,&Task1);
32   }
33 }
34
```

2 Blocking state and tick counter initialization

```
378
379 void MYRTOS_TaskWait(unsigned int NoTICKS, Task_ref* SlefTref){
380   SlefTref->TimingWaiting.Blocking = Enable;
381   SlefTref->TimingWaiting.Ticks_Count = NoTICKS;
382   //Task Should be blocked
383   SlefTref->TaskState = Suspend;
384
385   MYRTOS_OS_SVC_Set(SVC_TerminateTask);
386
387
388 }
389
```

3 Tick countdown and task reactivation

```
389
390 void MYRTOS_Update_TaskWaitingTime(){
391   for (int i= 0; i < OS_Control.NoOfActiveTacks; i++){
392     if(OS_Control.OSTasks[i]->TaskState == Suspend){
393       if (OS_Control.OSTasks[i]->TimingWaiting.Blocking == Enable){
394         OS_Control.OSTasks[i]->TimingWaiting.Ticks_Count--;
395         if(OS_Control.OSTasks[i]->TimingWaiting.Ticks_Count == 0){
396           OS_Control.OSTasks[i]->TimingWaiting.Blocking = Disable;
397           OS_Control.OSTasks[i]->TaskState = Waiting;
398           MYRTOS_OS_SVC_Set(SVC_TaskWaitingTime);
399         }
400       }
401     }
402   }
```

Figure 7. Original Keil captures with numbered labels: task delay request, blocking configuration, and tick-based wake-up logic.

Evidence B — idle path and execution traces

4 Idle task executes WFE while no task is runnable

```
37  unsigned char idleTackLED;
38
39  void MYRTOS_idleTack(){
40    while(1){
41      idleTackLED ^=1;
42      __asm volatile ("wfe");
43    }
44  }
45
46
```

5 Keil Logic Analyzer and System Analyzer observations



Figure 8. Original Keil captures: idle task WFE instruction and periodic task LED activity observed in the analyzers.

Observed result & engineering interpretation

The source-level debug captures show the intended suspension/countdown/reactivation logic. The analyzer traces show repeated task activity with idle-task activity also monitored, consistent with periodic waiting and scheduling. These captures support the functional scenario but do not, alone, establish exact delay accuracy, event wake-up source, power consumption, or freedom from timing races.

Verification note. The decrement operation occurs before the zero check; therefore the 100-tick behavior should be verified at the boundary in a dedicated watch-based test. WFE behavior depends on event/interrupt configuration. The screenshots document the observed session, not an exhaustive RTOS validation.

Test Case 04

Cascaded Priority-Based Preemption & Task Resumption

Objective. Demonstrate successive activation of higher-priority tasks and observe execution activity across four tasks. This scenario extends TC-02 by introducing multiple priority levels without repeating the underlying SVC, scheduler, or PendSV implementation details.

Configuration & execution sequence

Task	Priority (lower = higher)
Task1	4
Task2	3
Task3	2
Task4	1

Task1 toggles Task1LED and activates Task2 when Count reaches 100; it resets Count at 200. Task2 toggles Task2LED, activates Task3 when Count reaches 100, and requests its own termination when Count reaches 200. The described test continues the activation chain toward Task4 (highest priority). Once higher-priority tasks leave the runnable set, the scheduler can resume interrupted lower-priority tasks.

Evidence A — application-level activation chain

```
20
21 void task1(){
22   static int Count = 0;
23   while(1){
24     //Task1 Code
25     Task1LED ^=1;
26     Count++;
27     if(Count == 100){
28       MYRTOS_ActivateTask(&Task2);
29     }
30     if(Count == 200){
31       Count = 0;
32     }
33   }
34 }
35
```

```
76
77 void task4(){
78   static int Count = 0;
79   while(1){
80     //Task4 Code
81     Task4LED ^=1;
82     Count++;
83     if(Count == 3){
84       MYRTOS_AcquireMutex(&MUTEX2, &Task4);
85       MYRTOS_AcquireMutex(&MUTEX1, &Task4);
86     }
87     if(Count == 200){
88       Count = 0;
89       MYRTOS_ReleaseMutex(&MUTEX1);
90       MYRTOS_TerminateTask(&Task4);
91     }
92   }
93 }
```

Figure 9. Original Keil source captures: Task1 → Task2 activation; Task2 → Task3 activation and Task2 termination request.

Evidence B — execution trace

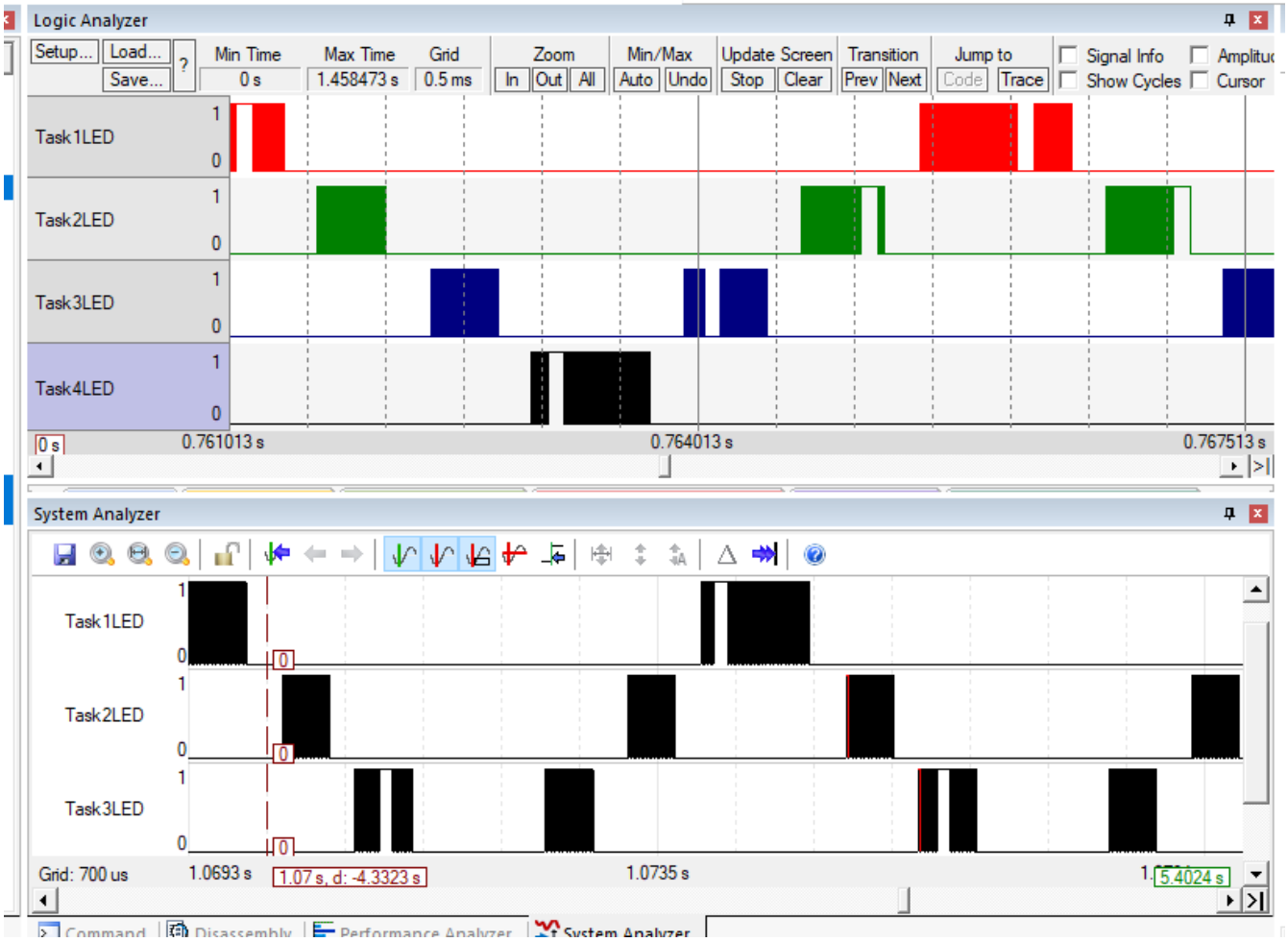


Figure 10. Keil Logic Analyzer and System Analyzer traces for Task1LED–Task4LED.

Observed outcome & limits

The screenshots document the application-side activation calls and show activity across all four task signals. The observed pattern is consistent with the intended cascaded-priority scheduling scenario. The available screenshots do not independently establish exact preemption instants, complete Task3/Task4 termination paths, or task-switch timing; those claims are not made here.

Test Case 05

Mutex Ownership, Task Blocking & Priority Inversion Scenario

Objective. Demonstrate mutex ownership and blocking behavior when a high-priority task attempts to acquire a mutex already owned by a lower-priority task. Inspect ownership transfer upon release and compare task activity in the Keil Logic Analyzer.

Test configuration

Priorities remain unchanged: Task1 = 4, Task2 = 3, Task3 = 2, Task4 = 1 (smaller number means higher priority). Task1 requests MUTEX1 at Count = 100 and releases it at Count = 200. Task4 attempts to acquire MUTEX1 at Count = 3; its shown code also requests release and termination at Count = 200.

Evidence A — acquisition and blocking

The source captures show Task1 acquiring MUTEX1 and activating Task2. MYRTOS_AcquireMutex() assigns CurrentTUser when the mutex is free. Otherwise, if NextTUser is empty, it records the requesting task as the waiter, sets its state to Suspend, and requests scheduling through SVC. Task4 calls this acquisition routine, providing the high-priority contention scenario.

1 Task1 requests mutex ownership

main.c* | Scheduler.c | MYRTOS_FIFO.c | CortexMX_OS_porting.c | core_cm3.h

```
22  unsigned char payload2[3] = {1,2,3};
23
24
25  // the comment to Try the Dead Lock State
26  void task1(){
27    static int Count = 0;
28    while(1){
29      //Task1 Code
30      Task1LED ^=1;
31      Count++;
32      if(Count == 100){
33        MYRTOS_AcquireMutex(&MUTEX1,&Task1);
34        MYRTOS_ActivateTask(&Task2);//
35      }
36      if(Count == 200){
37        Count = 0;
38        MYRTOS_ReleaseMutex(&MUTEX1);
39      }
40    }
41  }
42
43  void task2(){
44    static int Count = 0;
45    while(1){
```

2 Mutex acquisition: owner or waiting task

```
426
427  MYRTOS_errorID MYRTOS_AcquireMutex  (Mutex_ref* Mref,Task_ref* Tref){
428    if (Mref->CurrenrtTUser ==NULL){
429      Mref->CurrenrtTUser = Tref;
430    }else{
431      if(Mref->NextTUser == NULL)//not pending request by any other task
432      {
433        Mref->NextTUser = Tref;
434        //move to Suspend state until be released
435        Tref->TaskState = Suspend;
436        //to be Suspend immediately
437        MYRTOS_OS_SVC_Set(SVC_TerminateTask);
438      }else// there is a pending request by any other task
439      {
440        return MutexisReacedToMaxNumberOfUsers;
441      }
442    }
```

3 Task4 requests the occupied mutex

```
75  void task4(){
76    static int Count = 0;
77    while(1){
78      //Task4 Code
79      Task4LED ^=1;
80      Count++;
81      if(Count == 3){
82        MYRTOS_AcquireMutex(&MUTEX1, &Task4);
83      }
84      if(Count == 200){
85        Count = 0;
86        MYRTOS_ReleaseMutex(&MUTEX1);
87        MYRTOS_TerminateTask(&Task4);
88      }
89    }
90  }
```

Figure 11. Original Keil captures: Task1 mutex acquisition, MYRTOS_AcquireMutex() blocking logic, and Task4 acquisition request.

Evidence B — release and execution trace

The release implementation shown transfers CurrentTUser to NextTUser, clears NextTUser, changes the new owner state to Waiting, and requests an activation scheduling update. The Logic Analyzer shows activity on Task1LED through Task4LED. These captures illustrate the intended handoff and task activity; they do not independently timestamp mutex ownership or quantify inversion duration.

1 Mutex release and ownership transfer

```
445
446 MYRTOS_errorID MYRTOS_ReleaseMutex  (Mutex_ref* Mref){
447   if (Mref->CurrenrtTUser !=NULL){
448     Mref->CurrenrtTUser = Mref->NextTUser;
449      Mref->NextTUser = NULL;
450      Mref->CurrenrtTUser->TaskState = Waiting;
451      MYRTOS_OS_SVC_Set(SVC_ActivateTask);
452   }
453   return NOError;
454 }
```

2 Keil Logic Analyzer: four task signals

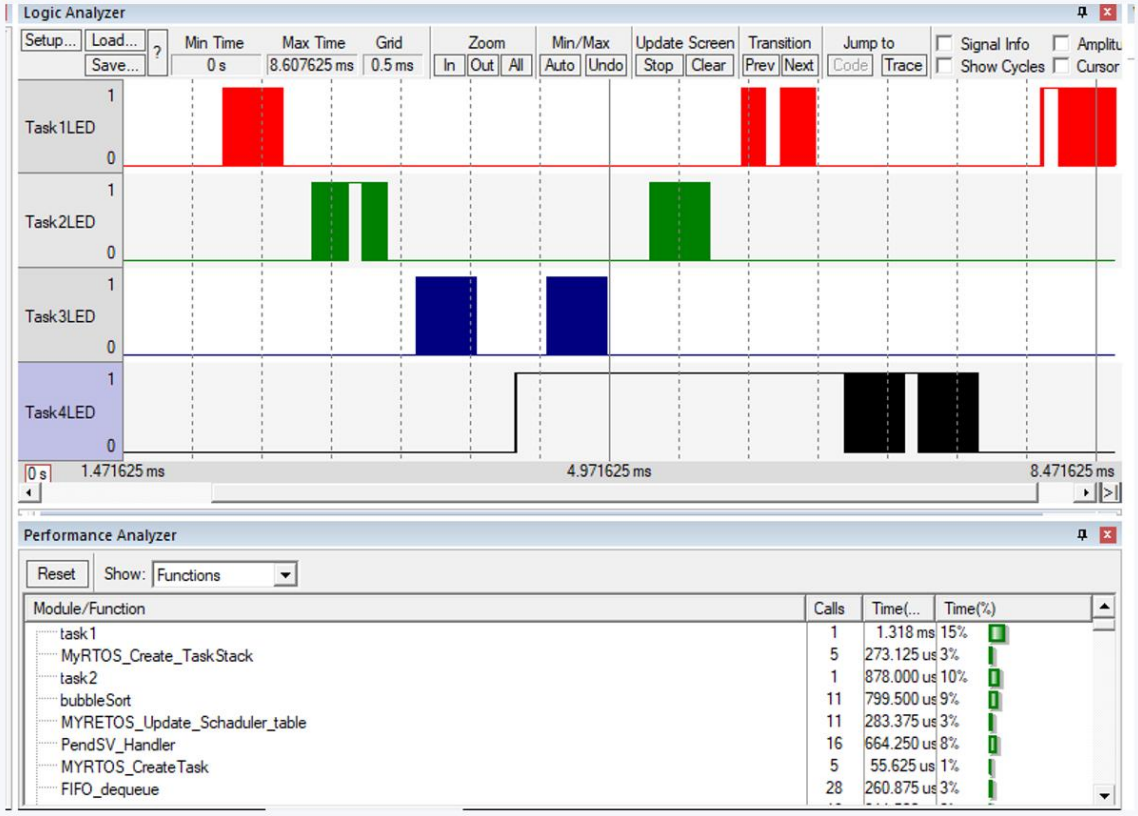


Figure 12. Original Keil captures: mutex release implementation and task LED execution traces.

Observed outcome & validation boundaries

The supplied source captures establish the mutex owner/waiter design and the application-side acquisition and release calls. The trace shows task activity consistent with the described test. The scenario illustrates the conditions for priority inversion: a high-priority task waits for a resource held by a lower-priority task, while medium-priority tasks may delay the owner. It does not demonstrate priority inheritance or a measured inversion interval.

Implementation review note. In the displayed MYRTOS_ReleaseMutex(), dereferencing CurrentTUser->TaskState after assigning NextTUser is unsafe when NextTUser is NULL. The routine also needs owner validation and a defined release-with-no-waiter path. These are review findings, not claims that the captured test failed. “Mutex” is the accurate term for this owner-based mechanism, rather than a counting semaphore.

Test Case 06

Circular-Wait Deadlock with Two Mutexes

Objective. Demonstrate a circular-wait deadlock when two tasks acquire two mutexes in conflicting orders, and observe the resulting loss of application task progress.

Test configuration and execution

Task1 has priority 4; Task4 has priority 1 (higher priority). Task1 first acquires MUTEX1, activates Task4, and then attempts MUTEX2. Task4 first acquires MUTEX2 and then attempts MUTEX1. Each task holds one mutex while requesting the mutex held by the other.

1 Task1: acquire MUTEX1, activate Task4, request MUTEX2

```
25  // the comment to Try the Dead Lock State
26  void task1(){
27    static int Count = 0;
28    while(1){
29      //Task1 Code
30      Task1LED ^=1;
31      Count++;
32      if(Count == 100){
33        MYRTOS_AcquireMutex(&MUTEX1,&Task1);
34        //MYRTOS_ActivateTask(&Task2);//
35        MYRTOS_ActivateTask(&Task4);
36        MYRTOS_AcquireMutex(&MUTEX2,&Task1);
37      }
38      if(Count == 200){
39        Count = 0;
40        MYRTOS_ReleaseMutex(&MUTEX1);
41      }
42    }
43  }
44
```

2 Task4: acquire MUTEX2, then request MUTEX1

```
76
77  void task4(){
78    static int Count = 0;
79    while(1){
80      //Task4 Code
81      Task4LED ^=1;
82      Count++;
83      if(Count == 3){
84        MYRTOS_AcquireMutex(&MUTEX2, &Task4);
85        MYRTOS_AcquireMutex(&MUTEX1, &Task4);
86      }
87      if(Count == 200){
88        Count = 0;
89        MYRTOS_ReleaseMutex(&MUTEX1);
90        MYRTOS_TerminateTask(&Task4);
91      }
92    }
93  }
```

Figure 13. Original Keil source captures, labeled to show opposing mutex acquisition orders.

Deadlock mechanism

Task	Mutex held	Mutex requested / result
Task1	MUTEX1	MUTEX2 → blocked
Task4	MUTEX2	MUTEX1 → blocked

The resulting wait-for cycle is Task1 → MUTEX2 → Task4 → MUTEX1 → Task1. Without a deadlock-prevention or recovery mechanism, neither blocked task can reach its intended release path. This is a task-level deadlock, not proof that the Cortex-M3 CPU or kernel interrupts stopped running.

Observed execution trace

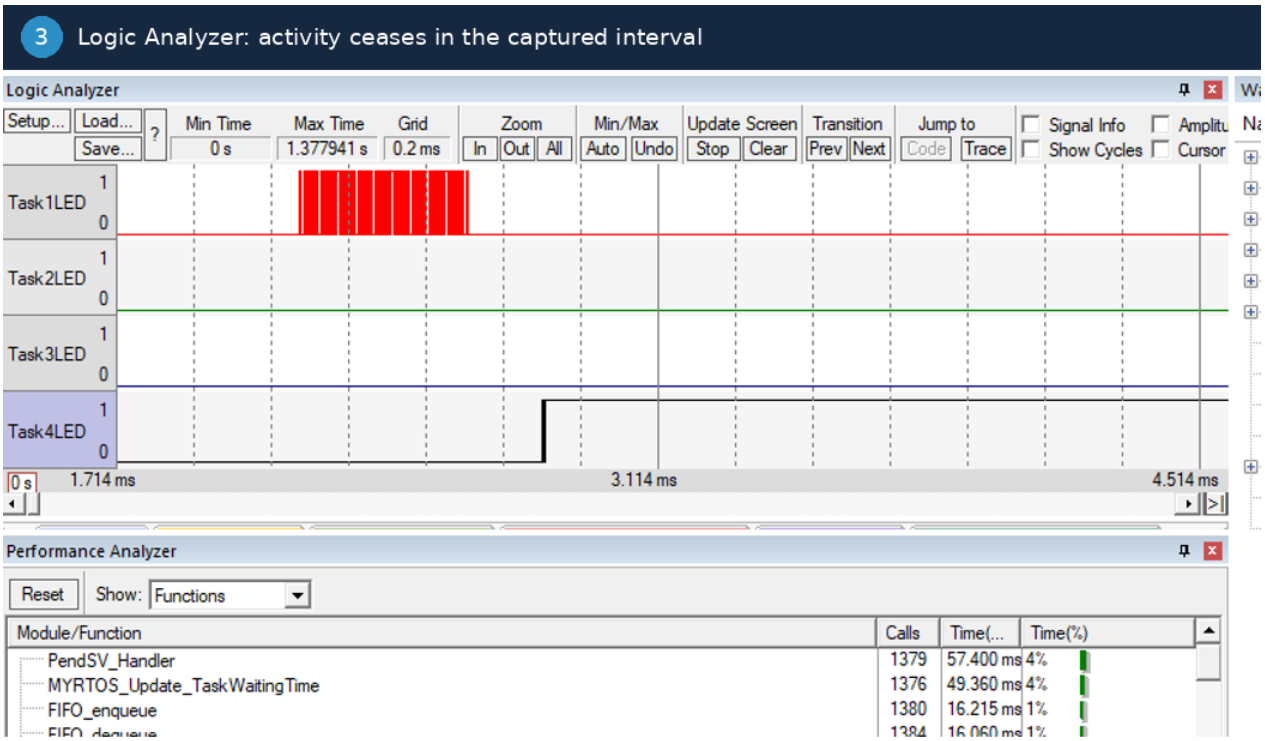


Figure 14. Keil Logic Analyzer evidence: task LED activity ceases within the shown observation window.

Observed result. The source snapshots establish the circular dependency, while the Logic Analyzer shows the absence of further task LED transitions in the captured interval. Together these observations are consistent with deadlock. Direct snapshots of both TaskState values and mutex owners would provide stronger runtime confirmation.

Engineering interpretation. The scenario intentionally exposes a synchronization limitation; it is not a passed deadlock-prevention test. Possible remedies include enforcing a global lock order, using bounded waits, or implementing deadlock detection/recovery. Priority inheritance addresses some priority-inversion effects but does not, by itself, break circular wait.

Consolidated Test Matrix

Case	Feature under test	Evidence-based assessment
TC-01	Task creation, PSP context, round robin	Execution and switching behavior observed
TC-02	Higher-priority activation / termination	Priority-directed execution supported by traces
TC-03	Tick-based wait and idle WFE	Delay/idle paths illustrated; timing and power not measured
TC-04	Cascaded priority preemption	Activation chain and task activity documented
TC-05	Mutex ownership and priority inversion	Blocking/ownership behavior illustrated; implementation caveat identified
TC-06	Two-mutex circular-wait deadlock	Deadlock-inducing code and stalled task traces documented

Engineering Assessment & Validation Boundaries

Demonstrated engineering scope. The project exercises Cortex-M3-specific execution contexts (MSP/PSP, privilege transitions, SVC, SysTick, and PendSV), manual task-stack preparation, priority and equal-priority scheduling, tick-based suspension, and basic mutex coordination. The report records functional scenarios using debugger and trace evidence rather than presenting a formal qualification campaign.

Important limitations. The available evidence does not establish worst-case interrupt latency, deterministic scheduling bounds, race freedom, complete register/context integrity across optimization settings, low-power current reduction, memory safety, or production suitability. In particular, the displayed mutex-release implementation can dereference a null successor when no waiter exists. The mutex design shown supports a single waiting task and does not demonstrate priority inheritance. The PendSV naked-handler implementation mixes C expressions and inline assembly and warrants compiler/disassembly-level review. Scheduler edge cases and stack capacity/bounds also require systematic testing.

Recommended follow-up verification. Add assertions for mutex owner/waiter state, deterministic tick-based tests, stack canaries and overflow checks, idle-only and all-blocked cases, compiler-optimization variants, exception-return validation, and explicit deadlock recovery/timeout scenarios. Repeat tests on physical STM32 hardware and record toolchain, clock, firmware revision, and test acceptance criteria.

Final Conclusion

The six cases form a coherent engineering demonstration of MYRTOS kernel mechanisms and the behavior of a custom Cortex-M3 scheduler. The work shows both successful execution paths and an intentionally reproduced deadlock failure mode. The evidence supports an educational/prototype kernel validation portfolio; further verification and corrective work are required before making reliability or production-readiness claims.